

07. Bank Transaction History System

Experimental Report: Real-Time Transaction Management and Data Structure Performance Evaluation

Group 07 --- CSD201
July 22, 2026

Abstract --- This report presents empirical performance evaluation and architectural analysis for the Bank Transaction History System (Group 07, CSD201 Project). The system models real-time banking operations under strict compliance rules: prohibiting direct record deletion or modification (preserving auditability via REVERSAL transactions) and enforcing strict overdraft prevention. We experimentally investigate three core Research Questions (RQ1, RQ2, RQ3) comparing four custom-implemented data structures---Doubly Linked List, Binary Search Tree (BST), Hash Table, and Sorted Linked List---on a scale of 100,000 transaction records across 1,000 randomized queries. Experimental results demonstrate that BST pruning provides superior performance for range queries (2.84× faster than Doubly Linked List), binary search on linked lists fails to achieve logarithmic efficiency due to pointer traversal overhead and $O(N^2)$ insertion costs, and multi-criteria filtering easily satisfies the 200 ms response time SLA threshold (achieving < 0.20 ms per query).

1. Introduction

Modern core banking systems handle millions of daily transactions, demanding both strict auditing integrity and sub-millisecond query performance. In accordance with banking regulations, historical ledgers must be immutable: transaction deletion or modification is strictly prohibited, requiring corrective actions to be recorded as distinct REVERSAL transactions. Furthermore, account balances must strictly prevent unauthorized overdrafts during withdrawal operations.

As transaction volume scales to 100,000 records and beyond, linear storage mechanisms become performance bottlenecks. This experimental study addresses three key Research Questions (RQs) to guide data structure selection for real-time transaction management:

- **Research Question 1 (RQ1):** Does a Doubly Linked List, Binary Search Tree (BST), or Hash Table yield the fastest execution time when querying transaction history by timestamp range over 100,000 records --- measured across 1,000 randomized range queries?
- **Research Question 2 (RQ2):** Is Binary Search superior to Linear Search when searching for transactions by ID on a Sorted Linked List, and does the high cost of maintaining sorted order upon new insertion negate any potential search advantage?
- **Research Question 3 (RQ3):** Which data structure most efficiently supports multi-criteria filtering (date range + transaction type + amount range) simultaneously while maintaining a strict response time SLA under 200 ms on a 100,000-record dataset?

To ensure stability and eliminate random execution noise, all experiments are conducted with a fixed random seed and pre-warmed JVM execution cycles.

2. Experimental Setup

The experimental environment evaluates custom Java implementations of data structures without relying on built-in Java Collection utilities (java.util.LinkedList, java.util.HashMap, etc.). The shared simulation parameters are detailed in Table 1.

Parameter	Value
Total Transaction Dataset Size (N)	100,000 records
Active Bank Accounts	10,000 accounts
Benchmark Repetitions (Q)	1,000 randomized queries

JVM Warmup Runs	500 query pre-warm cycles
Query SLA Response Threshold	< 200 ms per query
Audit Trail Integrity	Immutable log (Reversal transactions only)
Overdraft Prevention	Strict balance validation (balance >= amount)
Compared Data Structures	Doubly Linked List, Hash Table, BST, Sorted List

Table 1: Experimental setup and benchmarking configuration.

3. RQ1: Timestamp Range Query Performance

3.1 Experimental Design and Methodology

Range queries select all transactions occurring within a specified time window $[t_start, t_end]$. We measure the total execution time for 1,000 range queries on a dataset of 100,000 transactions across three data structures:

- **Custom Doubly Linked List:** Traverses all $N = 100,000$ nodes sequentially from head to tail ($O(N)$ lookup).
- **Custom Hash Table:** Performs a linear scan over internal bucket arrays to evaluate timestamp constraints for all entries ($O(N)$ scan).
- **Custom Binary Search Tree (BST):** Nodes are ordered by timestamp string. Range retrieval uses subtree pruning ($O(\log N + k)$), skipping branches whose subtrees fall entirely outside $[t_start, t_end]$.

3.2 Results --- Range Query Performance

<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Data Structure' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=9.5, greek=0, italic=0, link=[], rise=0, text='Data Structure', textColor=Color(0,0,0,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>		<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Avg Time / Query (μs)' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Hel vetica-Bold', fontSize=9.5, greek=0, italic=0, link=[], rise=0, text='Avg Time / Query (', textColor=Color(0,0, 0,1), us_lines=[]), ParaFrag(__tag__='b', bold=1, fontName='Hel vetica-Bold', fontSize=9.5, greek=0, italic=0, link=[], rise=0, text='μ', textColor=Col or(0,0,0,1), us_lines=[]), ParaFrag(__tag__='b', bold=1, fontName='Hel vetica-Bold', fontSize=9.5, greek=0, italic=0, link=[], rise=0, text='s)', textColor=Col or(0,0,0,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>	
<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Data Structure' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=9.5, greek=0, italic=0, link=[], rise=0, text='Data Structure', textColor=Color(0,0,0,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>		<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Avg Time / Query (μs)' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Hel vetica-Bold', fontSize=9.5, greek=0, italic=0, link=[], rise=0, text='Avg Time / Query (', textColor=Color(0,0, 0,1), us_lines=[]), ParaFrag(__tag__='b', bold=1, fontName='Hel vetica-Bold', fontSize=9.5, greek=0, italic=0, link=[], rise=0, text='μ', textColor=Col or(0,0,0,1), us_lines=[]), ParaFrag(__tag__='b', bold=1, fontName='Hel vetica-Bold', fontSize=9.5, greek=0, italic=0, link=[], rise=0, text='s)', textColor=Col or(0,0,0,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>	
Custom Doubly Linked List	292,636,100	292.64 μs	1.00x (Baseline)
Custom Hash Table	212,109,500	212.11 μs	1.38x faster
Custom Binary Search Tree (BST)	103,071,100	103.07 μs	2.84x faster

Table 2: RQ1 Empirical Benchmark Results (100,000 records, 1,000 range queries).

3.3 Analysis

As demonstrated in Table 2, the Custom BST achieves the fastest execution time, completing 1,000 range queries in 103.07 ms (103.07 μs per query). This represents a **2.84× speedup** over the Doubly Linked List and a **2.06× speedup** over the Hash Table.

The mathematical basis for this performance gap lies in the algorithmic structural properties:

- Hash Table Limitations:** Although a Hash Table provides $O(1)$ expected time for point lookups by transaction ID, hash functions destroy temporal ordering. Consequently, answering range queries requires iterating through all N elements across all hash buckets ($O(N)$ complexity).
- BST Pruning Advantage:** Because the BST maintains key order based on transaction timestamps, range traversal prunes subtrees where node values are strictly less than t_{start} (ignoring left subtrees) or strictly greater than t_{end} (ignoring right subtrees). This restricts traversal to relevant nodes ($O(\log N + k)$), dramatically reducing CPU comparison cycles.

3.4 Conclusion for RQ1

For time-range transaction history queries over 100,000 records, the **Binary Search Tree (BST)** is the optimal data structure, outperforming both Hash Tables and Doubly Linked Lists due to subtree range pruning.

4. RQ2: Binary Search vs. Linear Search on Sorted Linked List

4.1 Experimental Design and Hypotheses

RQ2 examines whether maintaining a sorted linked list enables efficient binary search by transaction ID. We test two specific hypotheses:

- **Search Efficiency Hypothesis:** Does Binary Search achieve superior lookup times compared to Linear Search on a sequential linked list?
- **Insertion Cost Hypothesis:** Does the cumulative $O(N^2)$ cost of sorted insertion negate any search performance advantage?

To measure insertion overhead accurately without CPU freeze, sorted insertion performance is evaluated on $N = 5,000$ records, followed by 1,000 transaction ID lookup queries.

4.2 Results --- Sorted Linked List Performance

Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Operation / Search Method' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=9.5, greek=0, italic=0, link=[], rise=0, text='Operation / Search Method', textColor=Color(0,0,0,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Total Execution Time (ns)' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=9.5, greek=0, italic=0, link=[], rise=0, text='Total Execution Time (ns)', textColor=Color(0,0,0,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Avg Time per Op (μs)' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=9.5, greek=0, italic=0, link=[], rise=0, text='Avg Time per Op (', textColor=Color(0,0,0,1), us_lines=[]), ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=9.5, greek=0, italic=0, link=[], rise=0, text='μ', textColor=Color(0,0,0,1), us_lines=[]), ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=9.5, greek=0, italic=0, link=[], rise=0, text='s)', textColor=Color(0,0,0,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph		
Sorted Insertion Cost ($O(N^2)$ total)	91,120,700 ns	18.22 μs / insert
Linear Search by Transaction ID	25,240,900 ns	25.24 μs / lookup
Binary Search on Linked List	15,279,000 ns	15.28 μs / lookup

Table 3: RQ2 Benchmark Results on Sorted Linked List ($N = 5,000$, 1,000 queries).

4.3 Analysis

While Binary Search appears marginally faster than Linear Search per lookup (15.28 μ s vs 25.24 μ s), the theoretical $O(\log N)$ divide-and-conquer advantage is fundamentally undermined by the linked list structure:

1. **Pointer Traversal Overhead:** Unlike contiguous arrays offering $O(1)$ random indexing, locating the midpoint node in a linked list requires traversing $N/2$ pointers ($O(N)$ node hops). As a result, Binary Search on a linked list incurs $O(N)$ overall time complexity per search.
2. **Dominant Insertion Overhead:** Maintaining sorted order during record ingestion requires comparing new transactions against existing nodes, resulting in $O(N^2)$ cumulative insertion time (91.12 ms for 5,000 records). In dynamic banking environments with frequent real-time transactions, this massive insertion penalty completely negates the minor lookup difference.

4.4 Conclusion for RQ2

Binary Search on a Sorted Linked List is **inefficient and impractical**. Pointer traversal prevents true $O(\log N)$ lookup scaling, and the severe $O(N^2)$ insertion overhead destroys operational feasibility. For ID lookups, a chained Hash Table ($O(1)$ expected lookup and $O(1)$ insertion) should be used instead.

5. RQ3: Multi-Criteria Filtering and SLA Compliance

5.1 Experimental Design and Hypotheses

Core banking dashboards require multi-criteria filtering where queries specify three criteria simultaneously: (1) Date interval $[t_start, t_end]$, (2) Transaction Type (DEPOSIT or WITHDRAWAL), and (3) Amount range $[amount_min, amount_max]$. We evaluate whether candidate data structures can process multi-criteria queries across 100,000 records while maintaining a strict Service Level Agreement (SLA) threshold of **< 200 ms per query**.

5.2 Results --- Multi-Criteria Benchmark and SLA Verification

Actual	Expected
<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Data Structure' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=9.5, greek=0, italic=0, link=[], rise=0, text='Data Structure', textColor=Color(0,0,0,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>	<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'SLA Compliance (< 200 ms)' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=9.5, greek=0, italic=0, link=[], rise=0, text='SLA Compliance (', textColor=Color(0,0,0,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>

Custom Doubly Linked List	173,050,600	0.173 ms	[OK] COMPLIANT
Custom Hash Table	202,443,300	0.202 ms	[OK] COMPLIANT
Custom BST (Pruning + Filtering)	123,140,200	0.123 ms	[OK] COMPLIANT

Table 4: RQ3 Multi-Criteria Filtering Benchmark and 200 ms SLA Verification (N = 100,000, 1,000 queries).

5.3 Analysis

All three data structures comfortably pass the 200 ms SLA requirement by a substantial margin:

- **Custom BST Performance:** Achieves the fastest filtering time at **0.123 ms per query** (123.14 ms total for 1,000 queries). The BST leverages temporal pruning to first isolate a small candidate subset of matching dates, then evaluates Type and Amount bounds only on that filtered subset.
- **Doubly Linked List & Hash Table:** Achieve 0.173 ms and 0.202 ms per query respectively. Even under full linear scans over 100,000 records, modern in-memory RAM execution speeds keep latency well below 1 ms.
- **SLA Margin:** The worst average query latency (0.202 ms) is nearly **1,000× faster** than the 200 ms threshold, confirming robust system performance under heavy query workloads.

5.4 Conclusion for RQ3

The **Custom BST** is the most effective data structure for multi-criteria transaction filtering. All evaluated structures strictly comply with the 200 ms SLA requirement for 100,000 records.

6. Overall Conclusions

1. **Range Queries (RQ1):** The **Binary Search Tree (BST)** is the optimal structure for timestamp range searches (2.84× faster than Doubly Linked List), as key ordering enables efficient subtree pruning.
2. **Sorted Linked List Lookups (RQ2):** Binary search on linked lists fails to achieve logarithmic performance due to sequential pointer traversals. The $O(N^2)$ insertion cost further renders sorted linked lists impractical for real-time transaction ingestion.
3. **Multi-Criteria Filtering & SLA (RQ3):** Combining BST date pruning with secondary criteria filtering yields the lowest latency (0.123 ms per query). All structures successfully satisfy the 200 ms SLA response threshold on 100,000 transaction records.